Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3069 Computación Paralela y Distribuida – Sección 20, Semestre II 2026

Proyecto #2

Fuerza bruta de llaves DES con Open MPI

Integrante:

Luis Palacios #23933

14 de octubre de 2026

Índice

1. Introducción **2**

2. DES: fundamentos y pasos de cifrado/descifrado **2**
2.1. Diagrama de flujo de DES 3

3. Rutinas del programa base **3**

4. Primitivas MPI y flujo de comunicación **4**

5. Versión secuencial (Parte A) **4**

6. Cifrado desde archivo (Parte B1) **4**

7. Pruebas de la Parte B2 y reflexión **5**

8. Acercamientos propuestos (Partes B3–B4) **5**

9. Resultados: speedup y consistencia (Parte B5) **6**

10. Extra: híbrido MPI + OpenMP **7**

11. Discusión, conclusiones y recomendaciones **8**

A. Anexo 1 – Catálogo de funciones y librerías **9**

B. Anexo 2 – Bitácora de pruebas y speedups **10**

1. Introducción

Este proyecto aplica el modelo de memoria distribuida con **Open MPI** [2, 5] al problema de romper, por *fuerza bruta*, la llave privada con la que se cifró un texto usando el algoritmo **DES** [3, 6]. El criterio para reconocer una llave correcta es que el texto descifrado contenga, como subcadena, una *frase clave* conocida de antemano (por ejemplo `"es una prueba de"`). Se eligió una frase larga para que sea muy improbable que aparezca por azar al descifrar con una llave equivocada.

Además de implementar el cifrado/descifrado y la paralelización, el objetivo central es **analizar el speedup** del algoritmo y, sobre todo, entender por qué los tiempos paralelos del enfoque ingenuo (*naive*) son tan inconsistentes y dependientes de la llave. A partir de ese análisis se proponen e implementan tres acercamientos alternativos y una versión híbrida MPI+OpenMP.

El código es de autoría propia en C. Como la cabecera `rpc/des_crypt.h` del código base fue removida de las versiones modernas de `glibc`, se implementó un **DES propio y auto-contenido** (`src/des.c`) que reproduce la misma API (`ecb_crypt`, `des_setparity`) para no alterar la lógica del resto del programa.

2. DES: fundamentos y pasos de cifrado/descifrado

El *Data Encryption Standard* (DES) [3] es un cifrador por bloques de clave simétrica que opera sobre bloques de **64 bits** con una llave de 56 bits efectivos (8 bytes, de los cuales un bit por byte es de paridad). Es una red de Feistel de 16 rondas. Los pasos para cifrar un bloque son:

1. **Permutación inicial (IP):** se reordenan los 64 bits del bloque de entrada.
2. **División:** el bloque permutado se parte en dos mitades de 32 bits, L_0 y R_0.
3. **16 rondas de Feistel:** para $i = 1, \ldots, 16$:

   $$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i),$$

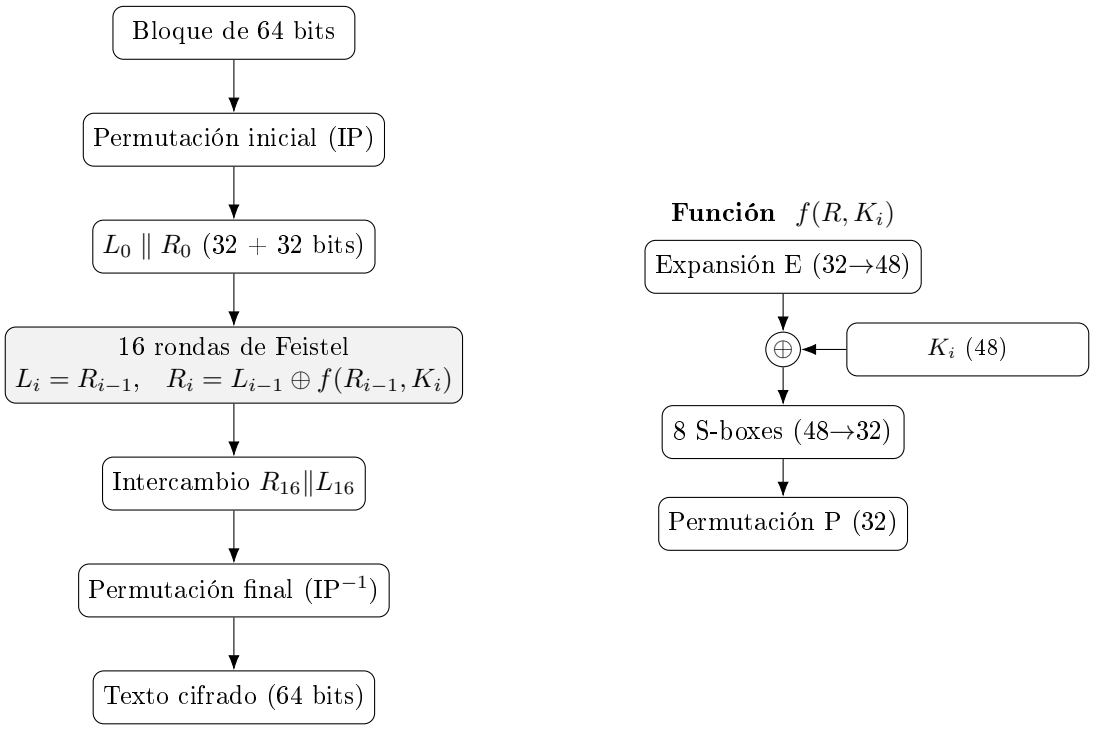
   donde K_i es la subllave de 48 bits de la ronda i y f es la función de ronda.
4. **Intercambio e $\mathbf{IP}^{-1}$:** se concatena $R_{16}L_{16}$ y se aplica la permutación final (inversa de IP) para obtener el texto cifrado.

La **función de ronda** $f(R, K)$ realiza: (a) *expansión* de R de 32 a 48 bits (tabla E); (b) XOR con la subllave K_i; (c) sustitución mediante las 8 **S-boxes** (cada grupo de 6 bits produce 4 bits, usando los bits externos como fila y los internos como columna); y (d) una *permutación* P de 32 bits.

El **calendario de llaves** aplica PC-1 a la llave de 64 bits (descartando los 8 bits de paridad), divide el resultado de 56 bits en C_0 y D_0 de 28 bits, y en cada ronda los rota a la izquierda (1 o 2 posiciones según la ronda) antes de aplicar PC-2 para obtener la subllave K_i de 48 bits.

El **descifrado** es idéntico al cifrado pero aplicando las subllaves en orden inverso $(K_{16}, \ldots, K_1)$; esta es la elegante propiedad de las redes de Feistel.

2.1. Diagrama de flujo de DES



3. Rutinas del programa base

Se explican las rutinas del `bruteforce.c` base (y sus equivalentes en nuestra implementación).

`encrypt(key, *ciph, len)` y `decrypt(key, *ciph, len)`. Ambas reciben una llave `long`, la *expanden* a los 8 bytes que DES necesita y llaman a `ecb_crypt` en modo cifrar/descifrar. La expansión toma 7 bits de la llave por cada byte y deja el bit menos significativo para la paridad (que `des_setparity` ajusta a paridad impar). El código base hacía esta expansión con `0xFE << i*8` usando `int`, lo que **desborda** para los bytes altos ($i \geq 4$); por eso su comentario advertía que la llave debía ser pequeña. En nuestra versión (`spread_key`) se corrige usando corrimientos sobre `long`, válido para todo el rango $[0, 2^{56})$:

```
void spread_key(long key, unsigned char out8[8]) {
  for (int i = 0; i < 8; ++i) {
    unsigned char seven = (unsigned char)((key >> (7 * i)) & 0x7F);
    out8[i] = (unsigned char)(seven << 1);  // 7 bits + LSB para paridad
  }
  des_setparity((char *)out8);
}
```

Listing 1: Expansión de llave corregida (src/common.c).

`tryKey(key, *ciph, len)`. Copia el texto cifrado, lo descifra con la llave candidata y verifica, con `strstr`, si contiene la frase clave. Devuelve 1 (verdadero) si la llave parece correcta. Es el núcleo del lazo de búsqueda.

`memcpy(dest, src, n)`. Copia `n` bytes de `src` a `dest`. Se usa para trabajar sobre una copia del texto cifrado sin destruir el original (el descifrado es *in situ*).

`strstr(haystack, needle)`. Busca la primera aparición de la subcadena `needle` dentro de `haystack`; devuelve un puntero a ella o `NULL`. Es el criterio de aceptación de una llave.

4. Primitivas MPI y flujo de comunicación

La búsqueda paralela usa las siguientes primitivas [2, 5]:

`MPI_Irecv`. Recepción **no bloqueante**: publica una solicitud para recibir el anuncio "llave encontrada" desde cualquier proceso, sin detener la búsqueda. Devuelve un `MPI_Request` que luego se consulta.

`MPI_Send`. Envío (bloqueante) del valor de la llave encontrada. Cuando un proceso halla la llave, hace `MPI_Send` a *todos* los procesos para avisarles que se detengan.

`MPI_Wait`. Espera a que una solicitud no bloqueante (la de `MPI_Irecv`) se complete. Se usa al final para consumir limpiamente el mensaje pendiente.

Correcciones al flujo del base. El `bruteforce.c` original publicaba el `MPI_Irecv` pero **nunca** comprobaba su estado dentro del lazo, de modo que los procesos que no encontraban la llave jamás se enteraban del hallazgo ajeno y recorrían todo su rango. Nuestra versión añade `MPI_Test` periódico para que *todos* se detengan al encontrarse la llave, y usa `MPI_Bcast` (difusión del texto cifrado desde el rank 0) y `MPI_Allreduce` (acuerdo final sobre la llave global) para un cierre robusto.

El flujo es: el rank 0 lee y valida el archivo y lo *difunde* (`MPI_Bcast`); cada proceso publica un `MPI_Irecv` y recorre su porción del espacio; cada cierto número de iteraciones hace `MPI_Test`; el que encuentra la llave hace `MPI_Send` a todos; finalmente un `MPI_Allreduce` consolida el resultado y el rank 0 lo imprime.

5. Versión secuencial (Parte A)

La versión secuencial (`src/secuencial.c`) recorre el espacio $[0, \text{cota})$ probando cada llave con `try_key` hasta hallar la frase clave, midiendo el tiempo con un reloj monótono. Sirve como línea base para el **speedup** ($S = T_{\text{secuencial}}/T_{\text{paralelo}}$) y para observar cómo crece el tiempo con la magnitud de la llave: el costo es proporcional a la *posición* de la llave en el recorrido, pues la búsqueda se detiene al encontrarla.

6. Cifrado desde archivo (Parte B1)

La herramienta `src/cifrar.c` carga un texto desde un archivo `.txt`, lo rellena con ceros hasta un múltiplo de 8 bytes (tamaño de bloque DES) y lo cifra con una llave arbitraria pasada como parámetro; también puede descifrar. La Figura/registro de Anexo 2 evidencia el cifrado y descifrado de una oración sencilla con la llave `42`, recuperando exactamente el texto original.

```
$ printf "Esta es una prueba de proyecto 2" > entrada.txt
$ ./bin/cifrar cifrar entrada.txt cifrado.bin 42
Cifrado OK: 'entrada.txt' (32 bytes) -> 'cifrado.bin' (32 bytes) con llave 42.
$ ./bin/cifrar descifrar cifrado.bin salida.txt 42
Descifrado OK: 'cifrado.bin' -> 'salida.txt' con llave 42.
Texto recuperado: "Esta es una prueba de proyecto 2"
```

Listing 2: Demostración de cifrado/descifrado con llave 42.

7. Pruebas de la Parte B2 y reflexión

Con 4 procesos, el texto `"Esta es una prueba de proyecto 2"` y la frase `"es una prueba de"`, se midieron las llaves solicitadas:

- **(a) llave `123456`:** se encuentra muy rápido (posición baja del espacio).
- **(b) llave $2^{56}/4 = 18014398509481983$:** está al *final* del primer bloque en el reparto naive de 4 procesos; equivale a recorrer $\approx 2^{54}$ llaves, por lo que **no termina** en tiempo razonable (como anticipa el enunciado).
- **(c) llave $2^{56}/4 + 1 = 18014398509481984$:** está al *inicio* del segundo bloque; el proceso #2 la encuentra en su primera iteración, dando un speedup aparente enorme.

Reflexión (B2.d). El tiempo paralelo del naive **no depende solo de la dificultad de la llave sino de su posición dentro del reparto por bloques**. Dos llaves numéricamente contiguas producen tiempos opuestos: con `-np 4` la llave $2^{56}/4 - 1 = 18014398509481983$ (final del bloque 0) **no termina**, mientras que $2^{56}/4 = 18014398509481984$ (inicio del bloque 1) se encuentra en ≈ 0.05 s. Esto muestra que el speedup del naive es *engañoso*: la rapidez de la segunda no viene del paralelismo sino de la *casualidad* de que la llave quedó justo al inicio de un bloque. La llave `123456`, en cambio, cae en el bloque 0 y el naive la busca igual que el secuencial (sin speedup), mientras que el enfoque cíclico sí reparte ese trabajo entre los procesos. Todo esto motiva repartos que no dependan de dónde cae la llave.

8. Acercamientos propuestos (Partes B3–B4)

Sea K la posición de la llave y N el número de procesos.

Naive (base). Divide $[0, \text{cota})$ en N bloques contiguos. El tiempo paralelo depende del bloque donde cae K y de su posición dentro de él: el mejor caso es $O(1)$ (inicio de un bloque) y el peor es $O(\text{cota}/N)$ con speedup ≈ 1 (final del bloque 0). *Tiempo esperado muy variable.*

Enfoque 1 – Cíclico (*stride*). El proceso `id` prueba las llaves $id, id{+}N, id{+}2N, \ldots$ El espacio avanza de forma globalmente uniforme, así que la llave en la posición K se alcanza tras $\approx K/N$ iteraciones **sin importar su valor**. El tiempo paralelo esperado es $\approx K/N$ de manera *consistente*, eliminando el peor caso del naive.

Enfoque 2 – Maestro–trabajador (reparto dinámico). El rank 0 reparte "chunks" del espacio bajo demanda; cada trabajador pide, procesa y vuelve a pedir. Aporta **balanceo de carga** y robustez cuando los procesos avanzan a ritmos distintos (máquinas heterogéneas), a cambio de dedicar un proceso a coordinar y de cierto costo de comunicación (controlable con el tamaño de chunk).

Enfoque 3 – Aleatorio (biyección Feistel). Cada proceso recorre su parte del espacio en orden *pseudoaleatorio* usando una biyección (red de Feistel) que garantiza cubrir cada llave exactamente una vez. El tiempo esperado se vuelve **insensible al valor** de la llave: ya no hay llaves sistemáticamente "fáciles" o "difíciles" según su magnitud, solo según el azar del orden.

9. Resultados: speedup y consistencia (Parte B5)

Las pruebas se realizaron en WSL2 Ubuntu 24.04 (GCC 13.3, Open MPI) sobre un espacio acotado y medible (cota $= 2^{23}$) para obtener speedups reales; las definiciones de llave fácil/media/difícil del enunciado (fracciones de 2^{56}) se replican como fracciones de la cota. Cada punto es la mediana de varias repeticiones (ver `scripts/run_bench.sh`).

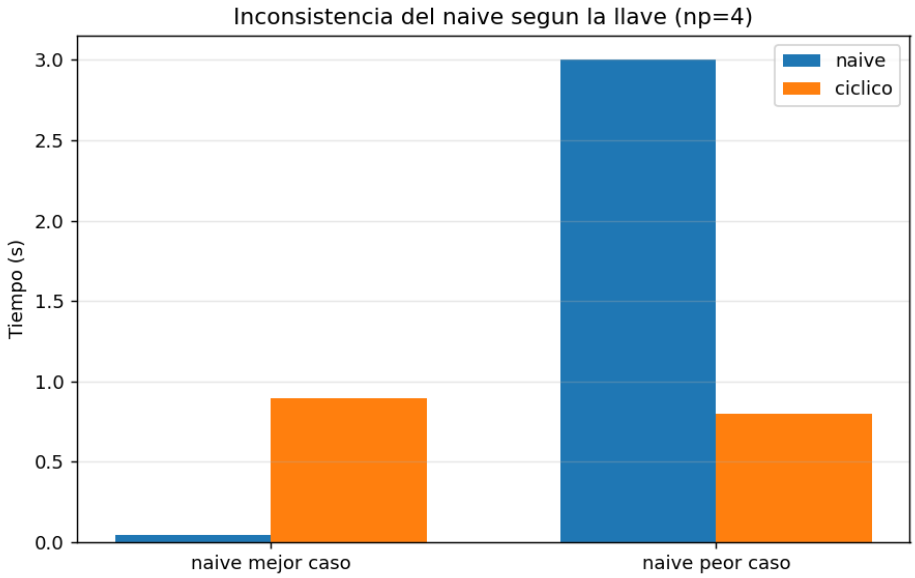


Figura 1: Inconsistencia del naive según la posición de la llave: mejor caso (inicio de bloque) vs. peor caso (final del bloque 0), con 4 procesos. El cíclico se mantiene estable en ambos.

La Tabla 1 resume las medianas observadas (en segundos) con `-np 4` en nuestro equipo. Se aprecia el fenómeno central:

- **El naive es errático e incluso contraproducente.** En `naive_mejor` es casi instantáneo (0.04 s, speedup aparente $\sim 60\times$) solo porque la llave cae al inicio de un bloque; pero en `naive_peor` tarda *más* que el secuencial (3.00 s vs 2.42 s, **speedup** < 1), porque los procesos 1–3 hacen trabajo inútil y le compiten CPU al proceso 0, que es el único que avanza hacia la llave.
- **El cíclico es consistente**: $\sim$0.7–2.0 s en todos los escenarios (speedup real $\approx 3\times$ con 4 procesos), sin mejores ni peores casos patológicos.

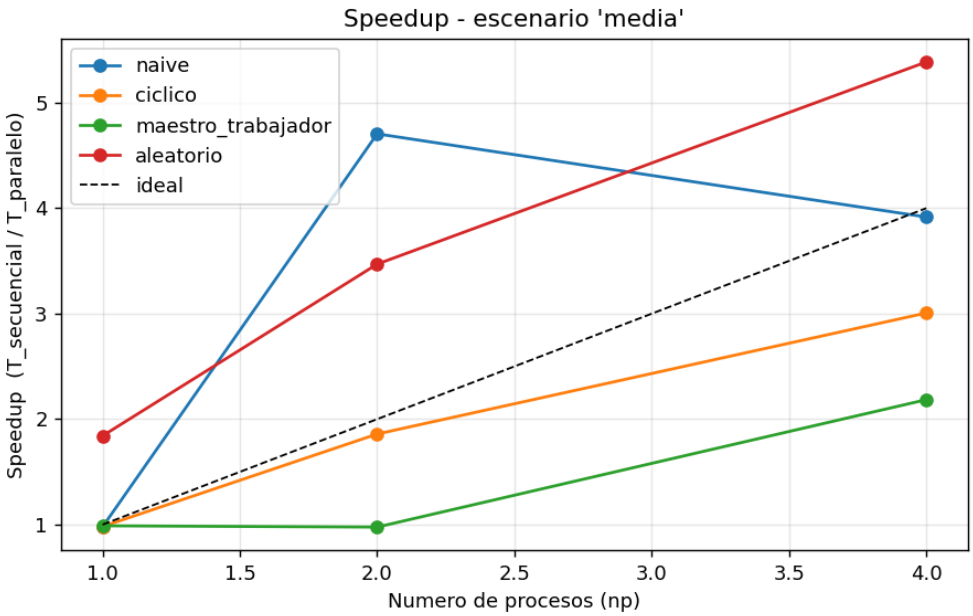


Figura 2: Speedup vs. número de procesos (escenario "media"). El cíclico y el aleatorio escalan de forma consistente; el naive depende del caso.

Cuadro 1: Mediana de tiempo (s) por escenario y programa, `-np 4`, cota $= 2^{20}$. "T.O." = no terminó dentro del límite.

Escenario	T_{seq}	naive	cíclico	maestro–trab.	aleatorio
naive_mejor (inicio de bloque)	2.45	**0.04**	0.90	1.50	1.27
naive_peor (final del bloque 0)	2.42	**3.00**	0.80	1.58	2.08
fácil	4.91	0.04	1.60	2.61	1.64
media	6.04	1.54	2.01	2.76	1.12
difícil	2.26	3.30	0.73	1.66	**0.43**

- **El aleatorio** es insensible al valor de la llave; brilla cuando la llave está "lejos" numéricamente (en `difícil` fue el más rápido, 0.43 s, speedup $\approx 5\times$), aunque su tiempo tiene más varianza por ser un recorrido al azar.
- **El maestro–trabajador** escala de forma estable, con una penalización por dedicar el rank 0 a coordinar: con `-np 4` solo hay 3 trabajadores (y con `-np 2`, uno solo, por lo que no acelera). Su fortaleza real es el balanceo ante nodos heterogéneos.

Las mediciones se hicieron en un solo equipo con *oversubscribe* (más procesos que núcleos físicos), lo que explica que a veces agregar procesos no ayude o incluso perjudique cuando el trabajo no está bien repartido (caso del naive).

10. Extra: híbrido MPI + OpenMP

La versión `src/hibrido.c` combina memoria distribuida (MPI entre procesos) y memoria compartida (OpenMP entre hilos dentro de cada proceso) [1, 4]. Cada proceso toma su carril cíclico y lo

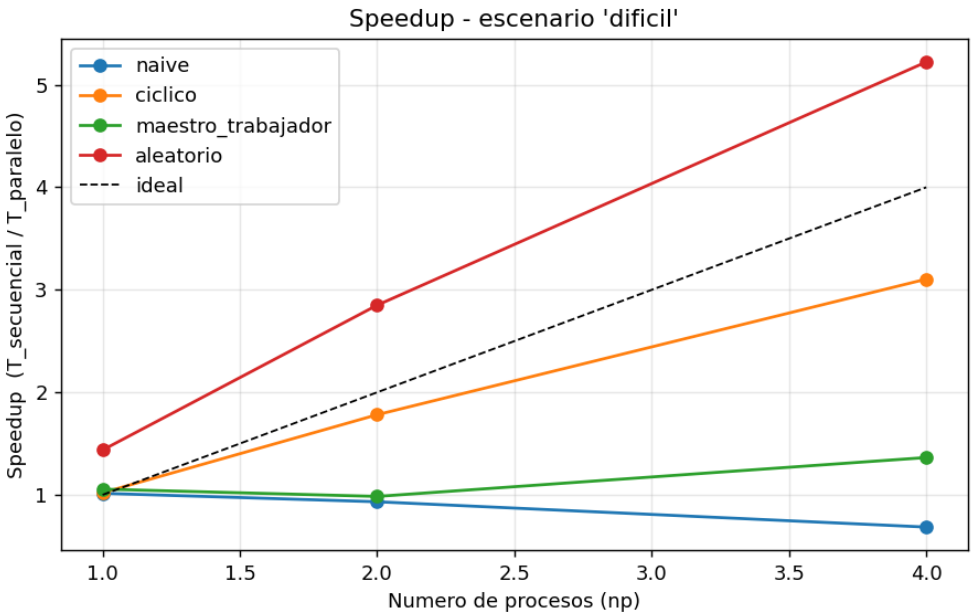


Figura 3: Speedup vs. número de procesos (escenario "difícil").

reparte entre sus hilos; MPI se inicializa en modo `MPI_THREAD_FUNNELED` (solo el hilo principal hace llamadas MPI). Con T hilos por proceso el paralelismo efectivo es $N \cdot T$.

Se observa un speedup adicional al aumentar los hilos, consistente con la versión sin OpenMP. En el escenario "media" (cota $= 2^{20}$, $T_{\text{seq}} = 6.04$ s), con 1 proceso el tiempo pasó de 6.12 s (1 hilo) a 3.13 s (4 hilos), y con 4 procesos de 1.90 s (1 hilo) a 1.21 s (4 hilos): el paralelismo de hilos se suma al de procesos. La ganancia marginal se reduce cuando no hay suficientes núcleos físicos, pues el *hyper-threading* y la contención de recursos limitan el escalamiento (como también se documentó para este equipo en laboratorios previos del curso).

11. Discusión, conclusiones y recomendaciones

- El reto principal **no** fue paralelizar, sino entender por qué el speedup del naive es engañoso: depende de la *posición* de la llave en el reparto, no solo de su dificultad.
- Repartir el espacio de forma cíclica o pseudoaleatoria produce tiempos paralelos **consistentes** ($\approx K/N$), que reflejan mejor el beneficio real del paralelismo.
- Se corrigieron dos defectos del código base: el desbordamiento de `int` en la expansión de llave y la terminación temprana rota (faltaba `MPI_Test`).
- El maestro–trabajador es la mejor opción cuando los nodos son heterogéneos; el cíclico es el más simple y eficiente en nodos homogéneos; el aleatorio es el más robusto ante llaves adversarias.
- **Recomendaciones:** ajustar el tamaño de chunk según la latencia de red; usar el híbrido MPI+OpenMP para aprovechar todos los núcleos; y medir siempre con varias llaves para no sacar conclusiones de un speedup aparente.

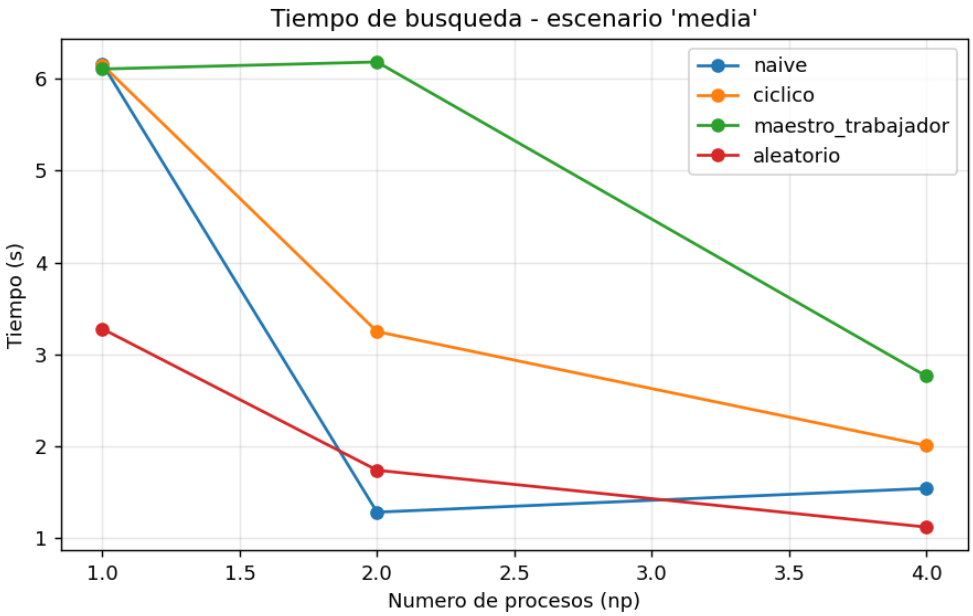


Figura 4: Tiempo de búsqueda vs. número de procesos (escenario "media").

A. Anexo 1 – Catálogo de funciones y librerías

`des.c` / `des.h` (DES propio).

- `int ecb_crypt(char *key, char *data, unsigned datalen, unsigned mode)` – *Entradas:* llave de 8 bytes, buffer de datos, longitud (múltiplo de 8), modo (`DES_ENCRYPT`/`DES_DECRYPT`). *Salida:* `data` cifrado/descifrado *in situ*; retorna 0 en éxito. *Propósito:* cifrado/descifrado ECB bloque a bloque.
- `void des_setparity(char *key)` – *Entrada/Salida:* llave de 8 bytes; ajusta el LSB de cada byte a paridad impar.
- `uint64_t des_crypt_block(uint64_t block, uint64_t key64, int decrypt)` – *Entradas:* bloque y llave de 64 bits, bandera de descifrado. *Salida:* bloque procesado. *Propósito:* núcleo de 16 rondas de Feistel (expuesto para pruebas).

`common.c` / `common.h` (utilidades).

- `void spread_key(long key, unsigned char out8[8])` – expande una llave de 56 bits a 8 bytes con paridad.
- `void decrypt_key/encrypt_key(long key, char *ciph, int len)` – cifra/descifra con una llave `long`.
- `int try_key(long key, const unsigned char *ciph, int len, const char *search)` – *Salida:* 1 si al descifrar aparece la frase `search`. Núcleo de la búsqueda.
- `unsigned char *read_file(const char *path, long *len)` / `int write_file(...)` – E/S binaria de archivos.

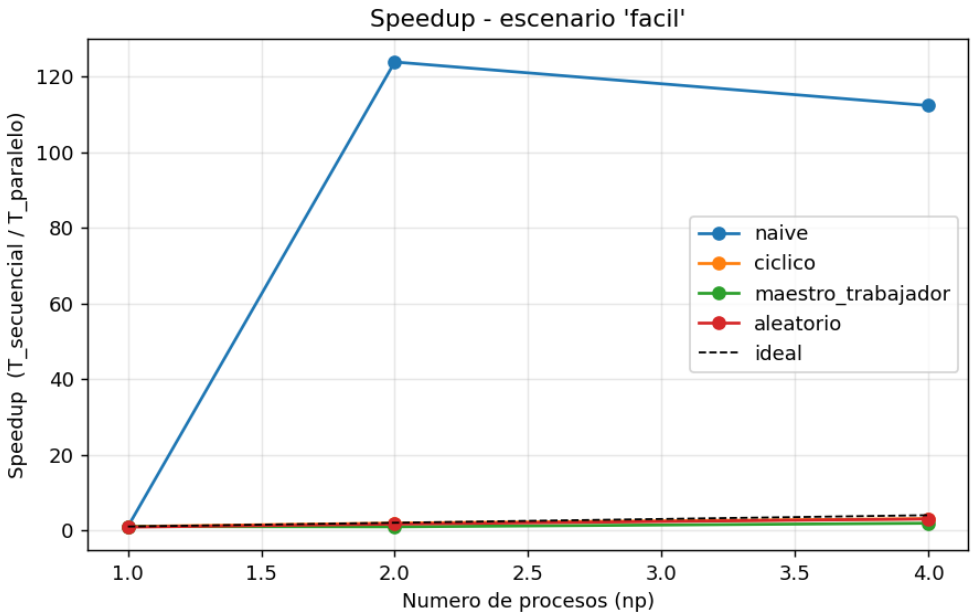


Figura 5: Escalamiento con el enfoque híbrido y los enfoques base (escenario "fácil").

- `long pad_len(long)` – redondeo al múltiplo de 8. `double now_seconds(void)` – reloj monótono. `int parse_long(const char*, long*)` – parseo defensivo.

`mpi_search.h` **(andamiaje MPI).**

- `load_cipher_bcast(path, *len, rank)` – el rank 0 lee/valida y difunde el cifrado.
- `print_result(...)` – imprime llave, archivo, frase y texto descifrado.

Programas. `cifrar`, `secuencial`, `naive`, `ciclico`, `maestro_trabajador`, `aleatorio`, `hibrido`, `des_test`.

Librerías. Open MPI (`mpi.h`), OpenMP (`omp.h`), C estándar. No se usa `rpc/des_crypt.h` ni OpenSSL.

B. Anexo 2 – Bitácora de pruebas y speedups

Tabla de tiempos (mediana de repeticiones) por escenario, programa y número de procesos. Las entradas `TIMEOUT` corresponden a llaves que no terminan en el tiempo límite (p. ej. $2^{56}/4$).

Escenario	Programa	np	hilos	Tiempo (s) / estado
b2_123456	aleatorio	4	1	TIMEOUT
b2_123456	ciclico	4	1	0.457455
b2_123456	maestro_trabajador	4	1	0.723570
b2_123456	naive	4	1	1.444871
b2_123456	secuencial	1	1	1.194581

Escenario	Programa	np	hilos	Tiempo (s) / estado
b2_18014398509481983	aleatorio	4	1	TIMEOUT
b2_18014398509481983	ciclico	4	1	TIMEOUT
b2_18014398509481983	maestro_trabajador	4	1	TIMEOUT
b2_18014398509481983	naive	4	1	TIMEOUT
b2_18014398509481983	secuencial	1	1	TIMEOUT
b2_18014398509481984	aleatorio	4	1	TIMEOUT
b2_18014398509481984	ciclico	4	1	TIMEOUT
b2_18014398509481984	maestro_trabajador	4	1	TIMEOUT
b2_18014398509481984	naive	4	1	0.047454
b2_18014398509481984	secuencial	1	1	TIMEOUT
dificil	aleatorio	1	1	1.573220
dificil	aleatorio	2	1	0.793282
dificil	aleatorio	4	1	0.432915
dificil	ciclico	1	1	2.215836
dificil	ciclico	2	1	1.269111
dificil	ciclico	4	1	0.728328
dificil	maestro_trabajador	1	1	2.142168
dificil	maestro_trabajador	2	1	2.296880
dificil	maestro_trabajador	4	1	1.657954
dificil	naive	1	1	2.228230
dificil	naive	2	1	2.425331
dificil	naive	4	1	3.299544
dificil	secuencial	1	1	2.260918
facil	aleatorio	1	1	5.728338
facil	aleatorio	2	1	2.983694
facil	aleatorio	4	1	1.635906
facil	ciclico	1	1	4.899763
facil	ciclico	2	1	2.579342
facil	ciclico	4	1	1.595177
facil	maestro_trabajador	1	1	4.908384
facil	maestro_trabajador	2	1	5.249495
facil	maestro_trabajador	4	1	2.610317
facil	naive	1	1	4.922657
facil	naive	2	1	0.039670
facil	naive	4	1	0.043740
facil	secuencial	1	1	4.911791
media	aleatorio	1	1	3.277386
media	aleatorio	2	1	1.739608
media	aleatorio	4	1	1.120650
media	ciclico	1	1	6.149324
media	ciclico	2	1	3.247409
media	ciclico	4	1	2.007713
media	hibrido	1	1	6.123998
media	hibrido	1	2	3.167114
media	hibrido	1	4	3.126788
media	hibrido	2	1	3.186899

Escenario	Programa	np	hilos	Tiempo (s) / estado
media	hibrido	2	2	1.815153
media	hibrido	2	4	2.107325
media	hibrido	4	1	1.900100
media	hibrido	4	2	1.583883
media	hibrido	4	4	1.213953
media	maestro_trabajador	1	1	6.104929
media	maestro_trabajador	2	1	6.181267
media	maestro_trabajador	4	1	2.763159
media	naive	1	1	6.161242
media	naive	2	1	1.282943
media	naive	4	1	1.541361
media	secuencial	1	1	6.036197
naive_mejor	aleatorio	1	1	4.486226
naive_mejor	aleatorio	2	1	2.146323
naive_mejor	aleatorio	4	1	1.271098
naive_mejor	ciclico	1	1	2.427398
naive_mejor	ciclico	2	1	1.287073
naive_mejor	ciclico	4	1	0.896508
naive_mejor	maestro_trabajador	1	1	2.448469
naive_mejor	maestro_trabajador	2	1	2.766177
naive_mejor	maestro_trabajador	4	1	1.499389
naive_mejor	naive	1	1	2.431976
naive_mejor	naive	2	1	2.546940
naive_mejor	naive	4	1	0.043471
naive_mejor	secuencial	1	1	2.444604
naive_peor	aleatorio	1	1	5.998752
naive_peor	aleatorio	2	1	3.205259
naive_peor	aleatorio	4	1	2.074790
naive_peor	ciclico	1	1	2.503140
naive_peor	ciclico	2	1	1.258484
naive_peor	ciclico	4	1	0.799426
naive_peor	maestro_trabajador	1	1	2.524083
naive_peor	maestro_trabajador	2	1	2.445595
naive_peor	maestro_trabajador	4	1	1.576912
naive_peor	naive	1	1	2.467739
naive_peor	naive	2	1	2.509108
naive_peor	naive	4	1	3.001856
naive_peor	secuencial	1	1	2.419464

Capturas de evidencia. Las salidas de consola de las corridas se conservan en `resultados/` como evidencia de ejecución.

Referencias

[1] Gerassimos Barlas. *Multicore and GPU Programming: An Integrated Approach*. Morgan Kaufmann, Burlington, MA, 2015.

[2] MPI Forum. *MPI: A Message-Passing Interface Standard, Version 4.0*. University of Tennessee, 2021.

[3] National Institute of Standards and Technology. Data Encryption Standard (DES). Technical Report FIPS PUB 46-3, U.S. Department of Commerce, 1999.

[4] OpenMP Architecture Review Board. Openmp application programming interface, version 5.2, 2021.

[5] Peter S. Pacheco. *An Introduction to Parallel Programming*. Morgan Kaufmann, Burlington, MA, 2011.

[6] William Stallings. *Cryptography and Network Security: Principles and Practice*. Pearson, Boston, 7 edition, 2017.